

Bioinformatics Analysis Report:

Attention is not all you need: pure attention loses rank doubly exponentially with depth

Paper Information

- **Paper ID:** 2103_03404
 - **Data Type:** Transformer model analysis, text sequences (Wikipedia biographies), synthetic sequences (sorting, convex hull)
 - **Organism:** Not applicable (theoretical machine learning study)
 - **Pages Analyzed:** 22
 - **Figures Extracted:** 0
-

Analysis Pipeline

Step 1: Data Preparation

Description: Prepare datasets for different experimental tasks to verify rank collapse phenomenon and path effectiveness across various domains

Tools: SentencePiece tokenizer, Custom data generation scripts, PyTorch/TensorFlow data loaders

Input: Raw text corpora (Wikipedia biographies, News articles), synthetic data for sorting and convex hull tasks

Output: Tokenized sequences with fixed length, formatted task-specific datasets with labels

Parameters: - `vocabulary_size` : 30522 - `max_sequence_length` : 128
- `num_samples_biographies` : 32 -

```
num_training_sequences_memorization : 500 -  
num_training_sequences_sorting : 1000 -  
num_test_sequences_sorting : 200 -  
num_training_sequences_convex_hull : 10000 -  
convex_hull_sequence_length : 10 - sorting_sequence_length : 8  
- alphabet_size_sorting : 10 - trajectory_radius_2d : 0.3 -  
num_points_per_trajectory : 1000
```

Example Command:

```
Not explicitly provided in paper. Would involve: sentencepiece --input=c
```

Best Practice Note: For tokenization, use consistent BPE/SentencePiece vocabulary across all experiments. For synthetic tasks, ensure sufficient data diversity and hold-out test sets to measure generalization vs memorization. Use teacher-forcing during training of recurrent single-layer models.

Step 2: Model Architecture Configuration

Description: Configure transformer architectures and create ablated variants (SAN, SAN+MLP, SAN+skip, full transformer) by selectively disabling components to isolate their effects on rank collapse

Tools: PyTorch/TensorFlow, Hugging Face Transformers library, Custom architecture modification scripts

Input: Pretrained transformer checkpoints (BERT, ALBERT, XLNet), randomly initialized models for training from scratch

Output: Modified model architectures with components disabled: pure SAN (no skip, no MLP), SAN+MLP, SAN+skip, full transformer

Parameters: - `models_tested` : ['BERT', 'ALBERT', 'XLNet'] -
`architectural_variants` : ['Pure SAN', 'SAN + MLP', 'SAN + skip connections', 'Full transformer'] - `depths_L` : [6, 12, 24, 32, 96] - `heads_H` : [3, 4, 12, 16, 96, 128] - `hidden_dimensions_d` : [32, 48, 84, 86, 128, 250] -
`attention_dimensions_dqk` : Not specified, varies by model

Example Command:

```
Not explicitly provided. Involves: model = BertModel.from_pretrained('be
```

Best Practice Note: When disabling components for ablation studies, maintain consistent hyperparameters (learning rate, batch size) across variants to isolate component effects. Properly handle parameter initialization when removing skip connections to avoid distribution shift during inference.

Step 3: Forward Pass and Residual Computation

Description: Execute forward pass through models and compute residual matrices at each layer to measure convergence to rank-1. Calculate the relative residual norm using $\ell_{1,\infty}$ composite norm to quantify rank collapse

Tools: Custom measurement scripts, NumPy/SciPy for matrix norm calculations, PyTorch/TensorFlow hooks for layerwise extraction

Input: Tokenized sequences (128-token excerpts), configured model architectures

Output: Residual norm values per layer: $\|\text{res}(\text{SAN}(X_l))\|_{1,\infty} / \|\text{SAN}(X_l)\|_{1,\infty}$ for each layer index l

Parameters: - `metric` : Relative residual norm: $\|\text{res}(\text{SAN}(X))\|_{1,\infty} / \|\text{SAN}(X)\|_{1,\infty}$ - `norm_definition` : $\|X\|_{1,\infty} = \sqrt{(\|X\|_1 * \|X\|_\infty)}$ - `residual_definition` : $\text{res}(X) = X - 1x^T$ where $x = \text{argmin}_x \|X - 1x^T\|$ - `num_test_samples` : 32 - `sequence_length` : 128 - `layer_indices_measured` : [0, 3, 6, 9, 12] - `runs_per_model` : 1

Example Command:

```
# Pseudo-code:
residuals = []
for layer_idx, output in enumerate(model.intermediate_outputs):
    X = output.last_hidden_states
    # Compute rank-1 approximation
    x = minimize_x_norm(X - ones @ x.T)
```

```
res = X - ones @ x.T
rel_norm = compute_l1inf_norm(res) / compute_l1inf_norm(X)
residuals.append(rel_norm)
```

Best Practice Note: Use consistent data distribution for evaluation.

Implement efficient computation of $\ell_{1,\infty}$ norm: $\|X\|_1 = \max_j \sum_i |X_{ij}|$, $\|X\|_\infty = \max_i \sum_j |X_{ij}|$. For deeper models ($L > 12$), measure at intermediate layers to capture exponential decay trend.

Step 4: Path Disentanglement and Effectiveness Analysis

Description: Isolate individual paths of varying lengths from trained models by extracting specific head sequences. Evaluate predictive power using only subsets of paths to verify that short paths carry most representational capacity

Tools: Custom path isolation scripts, Task-specific evaluation metrics (cross-entropy, accuracy)

Input: Trained transformer models, path decomposition weights P_{path} and W_{path} , test datasets for three tasks

Output: Performance metrics (token accuracy, sorting accuracy) for paths of lengths 0-6, averaged over multiple runs

Parameters: - `path_lengths_tested` : [0, 1, 2, 3, 4, 5, 6] -
`num_paths_sampled_per_length` : [5, 20] - `num_repetitions` : 5 -
`model_configurations` : {'memorization': {'L': 6, 'H': 2, 'd': 250}, 'sorting': {'L': 6, 'H': 2, 'd': 48}, 'convex_hull': {'L': 6, 'H': 3, 'd': 84}} -
`sampling_strategy` : Random sampling from all possible paths of given length

Example Command:

```
# Pseudo-code for path extraction:
def isolate_path_output(X, path_heads):
    # path_heads: list of head indices per layer, e.g. [2, 0, 5]
    output = X
    for layer, head in enumerate(path_heads):
        P = compute_attention_matrix(output, layer, head)
```

```

W = get_value_projection(layer, head)
output = P @ output @ W
return output

```

```

# Evaluate subset of paths
path_outputs = [isolate_path_output(X, path) for path in sampled_paths]
ensemble_output = sum(path_outputs) / len(path_outputs)

```

Best Practice Note: Path disentanglement requires careful implementation to correctly compute the product of attention matrices $P_{\text{path}} = P_L * \dots * P_1$ and weight matrices $W_{\text{path}} = W_1 * \dots * W_L$. Sample paths uniformly across all possible head combinations to avoid bias. The number of possible paths grows as H^L , so exhaustive evaluation is infeasible for large models.

Step 5: Inductive Bias Visualization

Description: Train single-layer transformer on 2D circular sequence prediction using teacher-forcing, then apply recurrently at inference to visualize convergence behavior and architectural biases

Tools: Custom training loop with teacher-forcing, Matplotlib/Plotting library, MSE loss function

Input: Synthetic 2D circular trajectories (two arcs), starting points $(-0.3, 0)$ and $(0.3, 0)$

Output: Visualization plots showing predicted trajectories for different hidden dimensions (32, 86, 128) and architectural variants

Parameters: - `training_method` : Teacher-forcing (ground truth coordinates fed as input at each step) - `inference_method` : No teacher-forcing (model output fed as next input) - `num_training_steps` : Not specified (trained until near-zero loss) - `hidden_dimensions_tested` : [32, 86, 128] - `architectural_variants_visualized` : ['No MLP, no skip', 'MLP, no skip', 'No MLP, skip'] - `num_inference_steps` : 1000

Example Command:

```

# Pseudo-code:
model = SingleLayerTransformer(hidden_dim=128, use_mlp=True, use_skip=False)
optimizer = Adam(model.parameters())
# Training loop with teacher forcing
for step in range(num_steps):
    pred = model(current_points) # current_points are ground truth
    loss = mse(pred, next_points)
    loss.backward(); optimizer.step()
# Inference without teacher forcing
for step in range(num_inference_steps):
    current_points = model(current_points) # Output becomes next input

```

Best Practice Note: Ensure model achieves near-zero training loss before inference visualization to isolate architectural effects from optimization issues. The recurrent application effectively simulates a multi-layer network without skip connections. Use MSE loss appropriate for regression of continuous coordinates.

Databases Used

- Wikipedia biographies dataset (Lebret et al. 2016) - for rank collapse verification
- Wikipedia and News corpora (Dagan et al. 2006, Wang et al. 2019) - for sequence memorization task
- Synthetic datasets for sorting task (random letter sequences) and convex hull prediction (random 2D points) - generated by authors

Key Methodological Findings

- Pure self-attention networks (SANs) without skip connections or MLPs converge to rank-1 matrices doubly exponentially with depth, causing token uniformity where all output tokens become identical
- Skip connections are crucial in preventing rank collapse by providing short paths (length 0) that preserve residual information; this reveals a previously unknown vital effect beyond facilitating optimization

- MLPs counteract rank collapse by increasing their Lipschitz constant, creating a tug-of-war with self-attention layers, though this comes at the cost of reduced robustness to input perturbations
- Layer normalization does not mitigate rank collapse as it only rescales features and cannot increase matrix rank
- Transformers behave like ensembles of shallow networks: short paths (length 1-2) carry most predictive power (>80% accuracy on memorization task), while long paths (length >3) perform near random
- Theoretical convergence bound requires $4\gamma\beta H < \sqrt{d_{\text{qk}}}$, but experiments show rank collapse occurs more broadly in practice, suggesting the theoretical condition is pessimistic
- Architectural variants like low-rank attention approximations may converge even faster to rank-1, while sparse attention patterns may slow convergence due to increased rank